

1. Title Page

Project Title: Fine-tuning a Multilingual Voice Cloning Model for Urdu

Author / Source: Synthesis from a technical consultation (Chat Conversation)

Date: 17 November 2025

Project Summary: A technical roadmap to enable high-fidelity, adaptive (zero-shot) voice cloning in the Urdu language by fine-tuning a state-of-the-art multilingual TTS model (Coqui XTTS).

2. Executive Summary

This project addresses a significant gap in speech technology: the lack of robust, publicly available Text-to-Speech (TTS) systems for Urdu that also support adaptive voice cloning. The initial problem analysis identified that while some Urdu TTS exists, no standard solutions offer a "cloning" architecture.

The core recommendation of this report is to *not* build a new model from scratch. Instead, the most efficient and powerful approach is to leverage a state-of-the-art, multilingual, zero-shot voice cloning model, such as **Coqui XTTS**. This architecture already possesses a powerful Speaker Encoder (the "cloning" component); the task is therefore reduced to "teaching" this existing model the Urdu language.

This fine-tuning process hinges on two key challenges:

1. **Data:** Acquiring a high-quality, multi-speaker, and meticulously transcribed Urdu audio dataset.
2. **Phonemization:** Solving the "tokenizer" problem by implementing a Grapheme-to-Phoneme (G2P) pipeline that converts Urdu script into a phonetic representation (IPA) that the model can understand.

This report provides a detailed, step-by-step roadmap for this objective. The immediate priorities are:

1. **G2P Solution:** Select and validate an Urdu G2P tool (e.g., `PronouncUR` or `epitran`).
2. **Data Auditing:** Acquire and rigorously audit the "Kaggle 20,000-file" or "HF urdu-tts" datasets for acoustic quality and transcript accuracy.
3. **Baseline Setup:** Establish a working Coqui XTTS environment and confirm its cloning capabilities in a supported language (e.g., English).

Following this roadmap will pivot the project from a high-risk research problem to a well-defined engineering and data-sourcing challenge, significantly increasing the probability of a successful Final Year Project (FYP).

3. Background & Context

The project's inception was marked by a critical problem: the core requirement for "adaptive voice cloning in Urdu" was blocked by a lack of available resources. The initial investigation confirmed that standard Urdu TTS models do not include the necessary cloning architecture.

The technical consultation concluded that attempting to build a novel cloning model for Urdu *ab initio* is impractical for an FYP. The recommended strategic pivot is to fine-tune a pre-trained, open-source, multilingual, zero-shot TTS model.

Recommended Model Architecture: Coqui XTTS

The analysis of this model's architecture is key to the project's strategy. The model is composed of several key components:

1. **Text Encoder / Phonemizer:** This is the primary target for fine-tuning. It is responsible for converting input text (e.g., "Hello") into a sequence of phonemes (e.g., $h \text{ } \partial \text{ } l \text{ } o \text{ } u$). This component must be taught to understand Urdu phonemes.
2. **Speaker Encoder:** This is the "voice cloning" component. It takes a 3-6 second audio clip of *any* voice and generates a "voice embedding" (a vector) that captures the unique timbre, pitch, and prosody of that speaker. This component is already highly effective and **must be frozen** during fine-tuning.
3. **Decoder / Vocoder:** This component takes the phoneme sequence (from the Text Encoder) and the voice embedding (from the Speaker Encoder) and synthesizes the final audio waveform. This component will be fine-tuned alongside the Text Encoder to learn the specific acoustic properties of the Urdu language.

By "freezing" the Speaker Encoder, we retain the model's powerful, pre-trained cloning ability. By "fine-tuning" the Text Encoder and Decoder, we teach this powerful cloner a new language: Urdu.

Appendix: Conversation Topic Index

This report is a synthesis of a technical consultation. The primary topics discussed included:

- Fine-tuning TTS for Urdu Voice Cloning
- Analysis of multilingual zero-shot TTS architectures
- Urdu Grapheme-to-Phoneme (G2P) resource discovery
- Urdu audio dataset discovery and auditing

4. Goals & Success Criteria

The primary goal is to produce an Urdu TTS model checkpoint that can, given a 3-6 second voice clip from *any* speaker, synthesize novel Urdu sentences in that speaker's voice.

Concrete Goals

1. **Intelligibility:** Generated Urdu speech must be fully intelligible to a native speaker.
2. **Cloning Fidelity:** The synthesized voice must be clearly identifiable as the target speaker.

3. **Naturalness:** The synthesized speech should have natural prosody (intonation and rhythm), avoiding a monotone, robotic sound.

Recommended Evaluation Metrics

To objectively measure success, the following metrics should be used:

- **Word Error Rate (WER):**
 - **What it is:** A measure of intelligibility.
 - **How to run:** Generate 100-200 audio clips from unseen text. Manually transcribe the generated audio (or use an Urdu ASR model if available). Compare the generated transcripts to the original text. $WER = (\text{Substitutions} + \text{Deletions} + \text{Insertions}) / \text{Total_Words}$.
 - **Success Target:** $WER < 15\%$.
- **Mean Opinion Score (MOS):**
 - **What it is:** A subjective score for overall quality and naturalness.
 - **How to run:** Create a survey with 10-20 generated samples. Have at least 10 native Urdu speakers rate each clip on a scale of 1 (Bad) to 5 (Excellent). The average score is the MOS.
 - **Success Target:** $MOS > 3.5$.
- **Speaker Similarity Score (SIM):**
 - **What it is:** A subjective score for cloning fidelity.
 - **How to run:** Present evaluators with the *original* 3-6s voice clip, followed by a 10s generated clip. Ask, "How similar is the synthesized voice to the original speaker?" (1-5 scale).
 - **Success Target:** $SIM > 4.0$.
- **ABX Test:**
 - **What it is:** A forced-choice test for fidelity.
 - **How to run:** Present an evaluator with the original voice clip (X), then two synthesized samples (A and B, where one is from your model and one is from a baseline). Ask, "Is A or B the same speaker as X?" This tests if your model is distinguishable from the original.

5. Detailed Roadmap

This roadmap provides an actionable, step-by-step process.

Phase 0: Setup & Prerequisites

1. **Hardware:** Secure a GPU with 24GB+ VRAM (e.g., NVIDIA 3090, 4090, A100). Google Colab Pro or a cloud VM is a viable alternative.
2. **Environment:** Create a Python virtual environment (e.g., `conda create -n tts_fyp python=3.10`).
3. **Repositories:**

- `git clone` the Coqui XTTS repository.
- Install all base dependencies (`pip install -r requirements.txt`).
- Install `tensorboard` for monitoring.

Phase 1: Model Selection & Rationale

1. **Model: Coqui XTTS-v2.**
2. **Checkpoint:** Download the latest pre-trained multilingual checkpoint.
3. **Strategy:** Freeze the Speaker Encoder. Fine-tune the Text Encoder, Decoder, and Vocoder.
4. **Baseline Test:** Run an inference in English using a 6-second clip of your own voice. Confirm the cloning functionality is working *before* any fine-tuning.

Phase 2: Data Collection & Preparation

This is the most time-consuming but critical phase.

1. Dataset Sizing:

- **Minimum (for test):** ~2 hours of *pristine* single-speaker audio.
- **Recommended (for FYP): 15-20 hours** of multi-speaker audio.
- **Ideal (for robust model):** 50+ hours.

2. Audio Quality (Non-Negotiable):

- **Sample Rate:** 22050Hz or 24000Hz (must match the model's training).
- **Format:** 16-bit, MONO, `.wav` file.
- **Noise:** Absolutely no background noise, reverb, or music.

3. Speaker Balance:

- Aim for a **50/50 gender balance** (e.g., 10h male, 10h female).
- An imbalanced dataset (e.g., 90% male) will bias the model, causing it to produce male-sounding voices even when cloning a female speaker.

4. Prosody Variation:

- Do not use only monotone readings. Include a mix of:
 - Declarative sentences (from news, Wikipedia).
 - Questions (interrogative).
 - Exclamations (excited, emotional). The `UrduSER` dataset is good for this.

5. Transcript Normalization:

- Transcripts must be **100% accurate** to the audio.
- Normalize all text: Remove English characters, spell out numbers in Urdu (e.g., "2" -> "دو"), and standardize punctuation.

6. Metadata Format:

- The model trainer requires a specific format, typically a `metadata.csv` file.
- Crucially**, the transcript must be converted to **phonemes** first (see Phase 3).
- Example** `metadata.csv` **line**: `audio_file_path|phonemic_transcript|speaker_name`
`./wavs/urdu_clip_0001.wav|m e: r a: n a: m d̤ i m n a: i h ε: |speaker_01`

Phase 3: Phonemization & Tokenizer Extension

This is the core technical challenge of adding a new language.

- The G2P Problem:** The model reads phonemes (sounds), not graphemes (script). You must convert all Urdu transcripts to their International Phonetic Alphabet (IPA) representation.
- G2P Tool Selection:** Choose one of the recommended tools:
 - `PronouncUR` (**GitHub**): A specialized Urdu-to-Phoneme tool. Likely more accurate.
 - `epitran` (**Python**): A multilingual tool that outputs IPA. Use the `urd-Arab` script code.
 - `phonemizer` (**Python**): A wrapper for `espeak-ng` which supports Urdu.
- Phonemization Pipeline:**
 - Input (Urdu Text):** "میرا نام ہے"
 - G2P Tool (`epitran`):** `epi.transliterate("میرا نام ہے", "urd-Arab")`
 - Output (IPA Phonemes):** `m e: r a: n a: m h ε:` (example)
- Tokenizer Extension:**
 - The model's `config.json` file contains a list of all phonemes it "knows."
 - You must **add your new Urdu IPA phonemes** (like `a:`, `ε:`, `d̤`) to this list.
 - Example** `config.json` **diff**:

```
{
  "phonemes": [
    "_", " ", "!", "\"", ",", "-", ".", ":", ";", "?",
    "a", "b", "c", ... "z", "ç", "ø", "œ", "ə", "ɛ",
    "g", "ɪ", "ɲ", "ɔ", "f", "ʊ", "ü", "v", "V", "Y",
    "æ", "ð", "ŋ", "χ", "β", "j", "ʔ", "ɜ", "i", "i:",
    + "a:", "d̤", "ε:"
    ...
  ],
  ...
}
```

Phase 4: Fine-Tuning Procedure

- Configuration:**

- Point your training config to your new `metadata.csv` (with phonemes) and the `config.json` (with added phonemes).
- **Batch Size:** Set as high as your VRAM allows (e.g., 8, 16).
- **Learning Rate (LR):** Start very low. This is a *fine-tune*. A good starting range is $1e-5$ to $5e-6$.

2. Freezing Components:

- Ensure your training script is configured to **freeze the speaker encoder**. Its weights must not be updated. This preserves the "cloning" capability.

3. Run Training:

- Launch the training script (e.g., `python train.py --config config.json`).

4. Monitoring:

- Run `tensorboard --logdir .` to watch the training loss.
- The training script will save checkpoints (e.g., every 1,000-5,000 steps).
- **Critically:** Listen to the synthesized audio samples generated at each checkpoint. At first, it will be babble. You should hear intelligible Urdu forming after 10k-20k steps.

Phase 5: Evaluation, Debugging & Iteration

1. **Listening Schedule:** At every 5,000 steps, synthesize and listen to 5 test sentences.

2. Common Failure Modes:

- **Garbled Phonemes:** The model is "confused." *Fix:* Check your G2P pipeline for errors. The phonemes in your `metadata.csv` may be wrong.
- **Gender Bias:** Cloning a female voice produces a male voice. *Fix:* Your dataset is imbalanced. Add more female speaker data.
- **Prosody Collapse (Monotone):** The model sounds robotic. *Fix:* Your dataset lacks prosodic variation. Add the emotional/interrogative clips (e.g., from `UrduSER`).

Phase 6: Deployment & Inference

1. **Inference Script:** Once you have a good checkpoint, write a simple script to use it.
2. **Example Inference (Pseudo-code):**

```
from TTS.api import TTS

# Load your fine-tuned model
tts = TTS(model_path="path/to/your/checkpoint.pth",
          config_path="path/to/your/config.json")

# The 6-second reference clip for cloning
reference_wav = "path/to/someones_voice.wav"

# The Urdu text to synthesize
```

```
urdu_text = "میرا نام جینائی ہے"

# Synthesize
tts.tts_to_file(text=urdu_text,
                speaker_wav=reference_wav,
                language="ur", # You may need to add 'ur'
                file_path="output.wav")
```

3. **Considerations:** Inference on CPU will be slow. For a real-time demo, GPU is required.

6. Resources & References

This section lists the specific resources identified during the consultation.

Grapheme-to-Phoneme (G2P) Tools

- 1. **PronouncUR (GitHub)**
 - Note:** A specialized "Urdu Pronunciation Lexicon Generator." Trained on a 39,000-word expert lexicon.
 - Link:** <https://github.com/harisbinzia/PronouncUR>
 - Next Action: Highly Recommended.** Clone this repo. Test its output. This is likely your most accurate G2P solution.
- 2. **epitran (Python Library)**
 - Note:** A multilingual G2P library that can output directly to IPA, which is ideal for XTTS.
 - Link:** <https://github.com/dmort27/epitran>
 - Next Action:** `pip install epitran`. Test it immediately with the `urd-Arab` script code. This is your fastest path to a working G2P pipeline.
- 3. **phonemizer (Python Library)**
 - Note:** A popular wrapper for the `espeak-ng` backend, which supports Urdu.
 - Link:** <https://github.com/bootphon/phonemizer>
 - Next Action:** A good fallback. `pip install phonemizer`. Test its output quality against `epitran`.

Audio Datasets

- 1. **Urdu Audio Dataset (20,000 files) (Kaggle)**
 - Note:** A large dataset described as "high-quality" with "authentic accents."
 - Link:** <https://www.kaggle.com/datasets/muhammadahmedansari/urdu-dataset-20000>
 - Next Action: Top Priority.** Download this dataset. Write a script to **audit** 100 random samples for audio quality (noise, mono, 22/24kHz) and transcript accuracy.
- 2. **urdu-tts (Hugging Face)**

- **Note:** Explicitly designed for TTS. Audio is pre-segmented based on silence.
- **Link:** <https://huggingface.co/datasets/muhammadsaadgondal/urdu-tts>
- **Next Action:** A strong alternative to the Kaggle dataset. Download and audit it in parallel.
- **3. UrduSER (Mendeley Data)**
 - **Note:** A dataset for Speech *Emotion* Recognition, using professional actors.
 - **Link:** <https://data.mendeley.com/datasets/jcpcfjnk5c2/3>
 - **Next Action: Supplemental Data.** Use this to add prosodic variation. Mix in 1-2 hours from this dataset to prevent your model from sounding monotone.
- **4. Rekhta Dataset (IEEE Paper)**
 - **Note:** A research paper describing a *methodology* for cleaning the noisy "Common Voice" Urdu dataset.
 - **Link:** <https://ieeexplore.ieee.org/document/10375044>
 - **Next Action:** Do not download. **Read this paper.** It will give you a clear process for cleaning your own data, which will be essential.

7. Code & Config Appendix

A. G2P Phonemizer Example (epitran)

```
#
#
# pip install epitran
#
import epitran

# Initialize Epitran for Urdu (Perso-Arabic script)
epi = epitran.Epitran('urd-Arab')

urdu_text = "میرا نام جیمنائی ہے"

# Transliterate to IPA
phonemes = epi.transliterate(urdu_text)

# Clean up spaces for the config
phoneme_list = phonemes.split(' ') # or whatever delimiter epitran uses

print(f"Text: {urdu_text}")
print(f"IPA: {phonemes}")
print(f"As List: {phoneme_list}")

# Example Output (varies):
```